ICS 202 – Data Structures and Algorithms

Topic 10: B-Trees

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Conventions (as in the slides): a B-tree of order m has at most $m-1$ keys per node; a node overflows when it holds m keys and is split, sending the middle key up (for even m the split is right-bias or left-bias, as stated). A non-root node must keep at least $\lceil m/2 \rceil - 1$ keys; on underflow a key rotation is tried before a merge. Trees are written level by level, left to right, e.g. root [20 40]; level 1: [5 10] [25 30] [50 60].

Part A: Multiple choice

Q1.

Unit 10 Activity 01 (T252)

Fill in the table for a B-tree of order 10: minimum number of keys in the root; maximum number of keys in the root; minimum number of subtrees of a non-root, non-leaf node; minimum number of keys in a non-root node. Which row is correct (in that order)?

(A) 1, 9, 5, 4

(B) 1, 10, 5, 5

(C) 0, 9, 2, 4

(D) 1, 9, 4, 5

Answer: (A). Root: 1 to $m-1=9$ keys; non-root: at least $\lceil 10/2 \rceil = 5$ subtrees and $\lceil 10/2 \rceil - 1 = 4$ keys.

My answer

Q2.

B-Trees Old Exams

For a non-empty B-tree of order m, what are the minimum number of keys and the minimum number of non-empty subtrees of a non-root (internal) node?

(A) $\lfloor m/2 \rfloor$ keys and $\lfloor m/2 \rfloor + 1$ subtrees

(B) $\lceil m/2 \rceil - 1$ keys and $\lceil m/2 \rceil$ subtrees

(C) 1 key and 2 subtrees

(D) $m-1$ keys and m subtrees

Answer: (B). Non-root nodes are kept at least half full: $\lceil m/2 \rceil$ children, hence $\lceil m/2 \rceil - 1$ keys.

My answer

Q3. B-Trees Old Exams

For the root node of a non-empty B-tree of order m, which list gives (maximum number of keys, maximum number of non-empty subtrees, minimum number of keys, minimum number of non-empty subtrees when the root is not a leaf)?

(A) m, $m+1$, 1, 2

(B) $m-1$, m, $\lceil m/2 \rceil - 1$, $\lceil m/2 \rceil$

(C) $m-1$, m, 0, 1

(D) $m-1$, m, 1, 2

Answer: (D). The root may hold as few as 1 key (2 children), and like every node at most $m-1$ keys and m children.

My answer

Q4. Quiz 4 Ver A/B (T241)

The minimum number of keys a B-tree of order 3 and height 2 can hold is

(A) 7

(B) 3

(C) 10

(D) 4

Answer: (A). Minimum keys $= 2\lceil m/2 \rceil^h - 1 = 2 \cdot 2^2 - 1 = 7$ (root at level 0).

My answer

Q5. Quiz 4 Ver A/B (T241)

The maximum number of keys a B-tree of order 3 and height 2 can hold is

(A) 13

(B) 20

(C) 25

(D) 26

Answer: (D). Maximum keys $= m^{h+1} - 1 = 3^3 - 1 = 26$.

My answer

Q6. Quiz 4 Ver A (T251)

The minimum number of keys a B-tree of order 5 and height 2 can hold is

(A) 9

(B) 12

(C) 17

(D) 26

Answer: (C). $2\lceil 5/2 \rceil^2 - 1 = 2 \cdot 9 - 1 = 17$.

My answer

Q7.

B-Trees Old Exams

What is the advantage of using B-trees over AVL trees?

(A) B-trees are balanced, while AVL trees are not balanced.

(B) B-trees require less memory than AVL trees.

(C) B-trees have a higher branching factor than AVL trees.

(D) Both A and C are advantages of using B-trees over AVL trees.

Answer: (C). Both are balanced; the large branching factor gives a much smaller height, so fewer disk accesses.

My answer

Q8.

AVL Trees and Heaps (T222)

Which of the following is true about AVL trees and B+ trees (the B-tree variant used for database indices)?

(A) AVL trees are height-balanced while B+ trees are not.

(B) AVL trees are used for external memory while B+ trees are used for internal memory.

(C) AVL trees have faster search times than B+ trees.

(D) B+ trees have faster search times than AVL trees.

Answer: (D). For large data on disk, the B-tree family needs far fewer disk accesses (about 4 vs. 24 for 16,000,000 keys in the slides); both are balanced.

My answer

Q9.

B-Trees Old Exams

What is the tight time complexity of searching for an element in a B-tree?

(A) $O(\log n)$

(B) $O(n)$

(C) $O(n \log n)$

(D) $O(1)$

Answer: (A). A B-tree is always balanced; search, insert and delete all take $O(\log n)$.

My answer

Q10.

Unit 10 Activity 01 (T252)

Consider the B-tree of order 5: root [49 75]; level 1: [7 20 31 40] [56 66 71] [81 89]; leaves: [2 4 6] [8 9 13 19] [23 25 26 30] [35 38] [42 44 48] (children of [7 20 31 40]), [53 54] [58 59 62 64] [67 68 70] [72 74] (children of [56 66 71]), [77 78 80] [84 85 87] [90 91 94 97] (children of [81 89]). Using linear search inside each node (algorithm `BTree-Search`), which keys are compared while searching for 68?

(A) 49, 56, 66, 71, 67, 68

(B) 49, 75, 71, 68

(C) 49, 75, 56, 66, 71, 67, 68

(D) 75, 71, 67, 68

Answer: (C). Root: $68 > 49$, $68 < 75 \Rightarrow$ child 1; $68 > 56$, $68 > 66$, $68 < 71 \Rightarrow$ child 2 = [67 68 70]; $68 > 67$, then $68 = 68$ found.

My answer

Q11.

[new]

In a B-tree of order 7, a node overflows after an insertion when it contains _____ keys, and a non-root node underflows after a deletion when it contains _____ keys.

(A) 7; 2

(B) 6; 3

(C) 7; 3

(D) 8; 2

Answer: (A). Overflow: $m = 7$ keys. Minimum is $\lceil 7/2 \rceil - 1 = 3$, so underflow means $\lceil m/2 \rceil - 2 = 2$ keys.

My answer

Q12.

Quiz 4 Ver A (T241)

Consider the B-tree of order 3 with 8 nodes and 11 keys: root [52]; level 1: [24 35] [200]; leaves: [3 10] [31] [50] [72 73] [900]. Insertion of which key will cause a node to split?

(A) 30

(B) 8

(C) 51

(D) 40

Answer: (B). 8 goes into the full leaf [3 10], giving [3 8 10] (3 keys = overflow). 30, 51 and 40 go into leaves with one key.

My answer

Q13. Quiz 4 Ver A (T251); B-Trees Old Exams

Consider the B-tree of order 5: root [27]; level 1: [15 20] [33 45]; leaves: [10 12 14] [18 19] [21 23 26] [29 31] [38 42] [47 50]. Insertion of which of the following sequences of keys, respectively, will result in a node split?

(A) 11, 16, 17, 22, 28, 40, 43, 46

(B) 8, 16, 25, 32

(C) 34, 35, 46

(D) 28, 30, 32

Answer: (D). 28, 30, 32 all go into [29 31]: [28 29 30 31 32] has 5 keys ⇒ split. In the other sequences no leaf exceeds 4 keys.

My answer

Q14. Unit 10 Activity 01 (T252)

The keys 78, 52, 81, 40, 33, 90, 85, 20, 38 are inserted in this order into an initially empty B-tree of order 3. The final tree is

(A) root [40]; level 1: [33] [78 85]; leaves: [20] [38] [52] [81] [90]

(B) root [33 78]; leaves: [20] [38 40 52] [81 85 90]

(C) root [78]; level 1: [38] [85]; leaves: [20 33] [40 52] [81] [90]

(D) root [78]; level 1: [33 40] [85]; leaves: [20] [38] [52] [81] [90]

Answer: (D). Inserting 38 overflows [20 33 38]; 33 moves up to [40], giving [33 40] under root [78] (same as the slides' example).

My answer

Q15. B-Trees Old Exams

The keys 24, 35, 28, 30, 25, 12, 15, 40, 5 are inserted, in the given order, into an empty B-tree of order 3. The final B-tree is

(A) root [24]; level 1: [12] [28 35]; leaves: [5] [15] [25] [30] [40]

(B) root [15 28]; leaves: [5 12] [24 25] [30 35 40]

(C) root [28]; level 1: [12 24] [35]; leaves: [5] [15] [25] [30] [40]

(D) root [28]; level 1: [15 24] [35]; leaves: [5 12] [25] [30] [40]

Answer: (C). After 40 the tree is [28] / [24] [35] / [12 15] [25] [30] [40]; inserting 5 splits [5 12 15], sending 12 up to give [12 24].

My answer

Q16.

Assignment 3 (T252)

A B-tree of order 3 is built by inserting 10, 20, 5, 6, 12, 30, 7, in order, into an initially empty tree. The final tree is

(A) root [10 20]; leaves: [5 6 7] [12] [30]

(B) root [6 20]; leaves: [5] [7 10 12] [30]

(C) root [10]; level 1: [6] [20]; leaves: [5] [7] [12] [30]

(D) root [10]; level 1: [6] [20]; leaves: [5] [7] [12 30]

Answer: (C). Before 7 the tree is [10 20] / [5 6] [12] [30]; 7 overflows [5 6 7] (6 goes up), then [6 10 20] overflows (10 goes up to a new root).

My answer

Q17.

Quiz 4 (T232); B-Trees Old Exams

The keys 13, 20, 18, 30, 24, 22, 23, 25 are inserted into an initially empty B-tree of order 5. The final tree is

(A) root [20 23]; leaves: [13 18] [22] [24 25 30]

(B) root [22]; leaves: [13 18 20] [23 24 25 30]

(C) root [20 24]; leaves: [13 18] [22 23] [25 30]

(D) root [18 24]; leaves: [13] [20 22 23] [25 30]

Answer: (C). 24 overflows [13 18 20 24 30] (20 up); 25 overflows [22 23 24 25 30] (24 up).

My answer

Q18.

Unit 10 Activity 01 (T252)

Insert 39 followed by 60 into the B-tree of order 3: root [18]; level 1: [13] [30 45]; leaves: [11] [16] [22 25] [41] [58 59]. The result is

(A) root [18]; level 1: [13] [30 45]; leaves: [11] [16] [22 25] [39 41] [58 59 60]

(B) root [18 45]; level 1: [13] [30] [59]; leaves: [11] [16] [22 25] [39 41] [58] [60]

(C) root [18 45]; level 1: [13] [30] [59]; leaves: [11] [16] [22 25] [39] [41] [58 60]

(D) root [30]; level 1: [18] [45 59]; leaves: [11 13 16] [22 25] [39 41] [58] [60]

Answer: (B). 39 joins [41]. 60 overflows [58 59 60]: 59 goes up, [30 45 59] overflows: 45 goes up into the root.

My answer

Q19.

Unit 10 Activity 01 (T252)

Insert 28 into the B-tree of order 5: root [49]; level 1: [7 20 31 40] [56 66 71 75]; leaves: [2 4 6] [8 9 13 19] [23 25 26 30] [32 34 35 38] [42 44 48] [53 54] [58 59 62 64] [67 68 70] [72 74] [77 78 80]. Which describes the result?

(A) root [49]; level 1: [7 20 26 31 40] [56 66 71 75]; leaves [23 25] and [28 30] replace [23 25 26 30]

(B) root [26 49]; level 1: [7 20] [31 40] [56 66 71 75]; leaves [23 25] and [28 30] replace [23 25 26 30]

(C) root [31 49]; level 1: [7 20 26] [40] [56 66 71 75]; leaves [23 25] and [28 30] replace [23 25 26 30]

(D) root [26 49]; level 1: [7 20] [31 40] [56 66 71 75]; leaves [23 25 28] and [30] replace [23 25 26 30]

Answer: (B). [23 25 26 28 30] overflows, 26 goes up; [7 20 26 31 40] overflows, 26 goes up to the root.

My answer

Q20.

Unit 10 Activity 01 (T252)

Insert 19 into the B-tree of order 4: root [30]; leaves: [10 20 25] [50 70], using **left-bias** splitting. The result is

(A) root [19 30]; leaves: [10] [20 25] [50 70]

(B) root [20 30]; leaves: [10 19] [25] [50 70]

(C) root [30]; leaves: [10 19 20 25] [50 70]

(D) root [25 30]; leaves: [10 19 20] [ ] [50 70]

Answer: (B). [10 19 20 25] overflows; left-bias keeps $m/2 = 2$ keys on the left, sends 20 up and leaves [25] on the right.

My answer

Q21.

Unit 10 Activity 01 (T252)

Insert 19 into the same B-tree of order 4 (root [30]; leaves: [10 20 25] [50 70]), but now using **right-bias** splitting. The result is

(A) root [20 30]; leaves: [10 19] [25] [50 70]

(B) root [19 25 30]; leaves: [10] [20] [ ] [50 70]

(C) root [19 30]; leaves: [10] [20 25] [50 70]

(D) root [30]; leaves: [10 19 20 25] [50 70]

Answer: (C). Right-bias: left gets $(m-1)/2 = 1$ key [10], 19 goes up, right gets $m/2 = 2$ keys [20 25].

My answer

Q22. Unit 10 Activity 02 (T252)

Consider the B-tree: root [30 60]; level 1: [20] [34 50] [70 78]; leaves: [5 17] [25] [32] [40 45] [53 55] [64 68] [75] [88 100]. Which gives the correct (predecessor, successor) of the keys 60, 20 and 70?

(A) 60: (55, 64); 20: (17, 25); 70: (68, 75)

(B) 60: (50, 70); 20: (5, 25); 70: (64, 78)

(C) 60: (55, 64); 20: (5, 30); 70: (68, 78)

(D) 60: (45, 64); 20: (17, 25); 70: (64, 75)

Answer: (A). Predecessor: one step left, then all the way right; successor: one step right, then all the way left.

My answer

Q23. Unit 10 Activity 02 (T252)

Delete 100 from the B-tree of order 4: root [102 140]; leaves: [90 100] [105 110] [152 186 194]. The result is

(A) root [102 140]; leaves: [90] [105 110] [152 186 194]

(B) root [105 140]; leaves: [90 102] [110] [152 186 194]

(C) root [140]; leaves: [90 102 105 110] [152 186 194]

(D) root [102 152]; leaves: [90] [105 110 140] [186 194]

Answer: (A). Order 4: minimum $\lceil 4/2 \rceil - 1 = 1$ key, so [90] does not underflow (case 1, direct deletion).

My answer

Q24. Unit 10 Activity 02 (T252)

Delete 118 from the B-tree of order 5: root [80 120 150]; children of 120 and 150 (left to right): [90 113 118] [126 135] [160 165 190] (the leftmost subtree, keys < 80, is not shown and not affected). What happens?

(A) 118 is removed directly; no underflow

90 113 underflows and a key is rotated from [126 135]

90 113 underflows and is merged with [126 135]

(B) 118 is replaced by its successor 120

Answer: (A). 118 is in a leaf, and [90 113] still has $2 = \lceil 5/2 \rceil - 1$ keys.

My answer

Q25.

Unit 10 Activity 02 (T252)

Delete 110 from the B-tree of order 5: root [102 140]; leaves: [90 100] [105 110] [152 186 194]. The result is

(A) root [140]; leaves: [90 100 102 105] [152 186 194]

(B) root [100 140]; leaves: [90] [102 105] [152 186 194]

(C) root [102 152]; leaves: [90 100] [105 140] [186 194]

(D) root [102 140]; leaves: [90 100] [105] [152 186 194]

Answer: (C). [105] underflows; the left sibling has only 2 keys but the right sibling has $3 \geq \lceil 5/2 \rceil$, so a left key-rotation: 140 down, 152 up.

My answer

Q26.

Unit 10 Activity 02 (T252)

Delete 135 from the B-tree of order 5: root [80 120 150]; children of 120 and 150: [90 113] [126 135] [160 165 190] (leftmost subtree, keys < 80, not shown). The root becomes

80 120 160 , with leaves [90 113] [126 150] [165 190]

80 113 150 , with leaves [90] [120 126] [160 165 190]

80 120 , with leaves [90 113] [126 150 160 165 190]

80 150 , with leaves [90 113 120 126] [160 165 190]

Answer: (A). [126] underflows; left sibling [90 113] is at the minimum, right sibling has 3 keys: left rotation (150 down, 160 up).

My answer

Q27.

Unit 10 Activity 02 (T252)

Delete 135 from the B-tree of order 5: root [80 120 150]; children of 120 and 150: [90 113 118] [126 135] [160 165] (leftmost subtree, keys < 80, not shown). The result is

(A) root [80 120]; leaves: [90 113 118] [126 150 160 165]

(B) root [80 118 150]; leaves: [90 113] [120 126] [160 165]

(C) root [80 120 160]; leaves: [90 113 118] [126 150] [165]

(D) root [80 150]; leaves: [90 113 118 120 126] [160 165]

Answer: (B). [126] underflows; the left sibling has 3 keys, so a right key-rotation (118 up, 120 down). A rotation must be tried before merging.

My answer

Q28.

Unit 10 Activity 02 (T252)

Delete 105 from the B-tree of order 5: root [102 140]; leaves: [90 95 100] [105 110] [152 186]. The result is

(A) root [102]; leaves: [90 95 100] [110 140 152 186]

(B) root [140]; leaves: [90 95 100 102 110] [152 186]

(C) root [110 140]; leaves: [90 95 100] [102] [152 186]

(D) root [100 140]; leaves: [90 95] [102 110] [152 186]

Answer: (D). [110] underflows; the left sibling has 3 keys, so a right rotation: 100 up, 102 down.

My answer

Q29.

Unit 10 Activity 02 (T252)

Delete 86 from the B-tree of order 5: root [40 80 110]; children of 80 and 110: [50 73 78] [86 95] [120 125 150] (leftmost subtree, keys < 40, not shown). If the missing key is borrowed from the **left** sibling, the result is

(A) root [40 78 110]; leaves: [50 73] [80 95] [120 125 150]

(B) root [40 80 120]; leaves: [50 73 78] [95 110] [125 150]

(C) root [40 110]; leaves: [50 73 78 80 95] [120 125 150]

(D) root [40 73 110]; leaves: [50] [78 80 95] [120 125 150]

Answer: (A). Right key-rotation: max of the left sibling (78) goes up, separator 80 goes down into [95].

My answer

Q30.

Unit 10 Activity 02 (T252)

Delete 110 from the B-tree of order 5: root [55 95 128]; children of 95 and 128: [65 88 93] [101 110] [135 140 155] (leftmost subtree, keys < 55, not shown). Which of the following are possible results?
I. root [55 93 128]; leaves: [65 88] [95 101] [135 140 155]
II. root [55 95 135]; leaves: [65 88 93] [101 128] [140 155]

(A) I only

(B) II only

(C) Both I and II

(D) Neither; the leaf must be merged with a sibling

Answer: (C). Case 4: both adjacent siblings have $3 \geq \lceil 5/2 \rceil$ keys, so either a right rotation (I) or a left rotation (II) is valid.

My answer

Q31. Unit 10 Activity 02 (T252)

Delete 100 from the B-tree of order 5: root [102 140]; leaves: [90 100] [105 110] [152 186 194]. The result is

(A) root [105 140]; leaves: [90 102] [110] [152 186 194]

(B) root [140]; leaves: [90 102 105 110] [152 186 194]

(C) root [102 152]; leaves: [90 105 110] [140] [186 194]

(D) root [102 140]; leaves: [90] [105 110] [152 186 194]

Answer: (B). [90] underflows; its only adjacent sibling [105 110] has $2 = \lceil 5/2 \rceil - 1$ keys, so merge [90], 102, [105 110]. The root keeps 1 key (root does not underflow).

My answer

Q32. Unit 10 Activity 02 (T252)

Delete 472 from the B-tree of order 5: root [395 430 480]; leaves: [380 382] [406 412] [451 472] [493 506]. If the merge is done with the **right** sibling, the result is

(A) root [395 480]; leaves: [380 382] [406 412 430 451] [493 506]

(B) root [395 430 493]; leaves: [380 382] [406 412] [451 480] [506]

(C) root [395 430]; leaves: [380 382] [406 412] [451 480 493 506]

(D) root [430]; leaves: [380 382 395 406 412] [451 480 493 506]

Answer: (C). [451] underflows, both siblings have the minimum 2 keys, so merge [451], separator 480 and [493 506]. ((A) is the result of merging with the left sibling.)

My answer

Q33. Quiz 4 Ver A (T241); Quiz 4 V1 (T251)

Consider the B-tree of order 3: root [52]; level 1: [24 35] [200]; leaves: [3 10] [31] [50] [72 73] [900]. Deletion of which key will certainly lead two nodes to merge into one node?

(A) 52

(B) 200

(C) 31

(D) 50

Answer: (D). [50] empties and its only adjacent sibling [31] has the minimum 1 key, so it must merge. 52 merges only if replaced by its predecessor 50 (its successor 72 causes no merge); 200 and 31 are fixed without merging. (The T251 key said "more than one answer".)

My answer

Q34.

Quiz 4 Ver A (T241)

For the same B-tree of order 3 (root [52]; level 1: [24 35] [200]; leaves: [3 10] [31] [50] [72 73] [900]), deletion of the key 900 will result in a B-tree with

(A) height 1

(B) 7 nodes in total

(C) 8 nodes in total

(D) 6 nodes in total

Answer: (C). [900] empties; the left sibling [72 73] can lend: 73 goes up, 200 goes down. Result [52] / [24 35] [73] / [3 10] [31] [50] [72] [200]: still 8 nodes.

My answer

Q35.

Quiz 4 Ver B (T241)

For the same B-tree of order 3 (root [52]; level 1: [24 35] [200]; leaves: [3 10] [31] [50] [72 73] [900]), deletion of the key 24 will result in a B-tree with

(A) height 1

(B) 8 nodes in total

(C) 7 nodes in total

(D) 6 nodes in total

Answer: (B). With the predecessor, 10 replaces 24 and [3] remains; with the successor, 31 replaces 24 and a rotation from [3 10] fixes the leaf. Either way: 8 nodes.

My answer

Q36.

B-Trees Old Exams

Delete 27 from the B-tree of order 3: root [30 56]; level 1: [24] [36] [68]; leaves: [22] [27] [32 34] [40 52] [66] [70]. The final tree is

(A) root [30 56]; level 1: [22 24] [36] [68]; leaves: [32 34] [40 52] [66] [70]

(B) root [56]; level 1: [30 36] [68]; leaves: [22 24] [32 34] [40 52] [66] [70]

(C) root [36]; level 1: [30] [56 68]; leaves: [22 24] [32 34] [40 52] [66] [70]

(D) root [56]; level 1: [30 36] [68]; leaves: [22 24] [32 34] [40 52] [66 70]

Answer: (B). [27] empties, sibling [22] is minimal: merge [22 24]. Now [24]'s node is empty and sibling [36] is minimal: merge (empty), 30, [36] into [30 36]; root keeps [56].

My answer

Q37. [new]

Delete 40 from the B-tree of order 3: root [12]; level 1: [4 8] [20]; leaves: [2] [6] [10] [15] [40]. The final tree is

(A) root [8]; level 1: [4] [12]; leaves: [2] [6] [10] [15 20]

(B) root [12]; level 1: [4 8] [15]; leaves: [2] [6] [10] [20]

(C) root [8 12]; leaves: [2 4 6] [10] [15 20]

(D) root [12]; level 1: [4 8] [ ]; leaves: [2] [6] [10] [15 20]

Answer: (A). [40] empties; [15] is minimal, so merge [15 20]. The parent is now empty; its left sibling [4 8] can lend: 8 goes up, 12 comes down, and 8's right subtree [10] becomes the left child of 12.

My answer

Q38. Quiz 4 Ver A (T242)

Delete the record with key 90 from the B-tree of order 4: root [50 70]; level 1: [33] [65] [90]; leaves: [22] [40] [53 56] [67 68] [75] [92]. The final tree is

(A) root [50 70]; level 1: [33] [65] [92]; leaves: [22] [40] [53 56] [67 68] [75] [ ]

(B) root [65]; level 1: [33 50] [70]; leaves: [22] [40] [53 56] [67 68] [75 92]

(C) root [50]; level 1: [33] [65 70]; leaves: [22] [40] [53 56] [67 68] [75] [92]

(D) root [50]; level 1: [33] [65 70]; leaves: [22] [40] [53 56] [67 68] [75 92]

Answer: (D). 90 is replaced by 92 (or 75); the leaf empties and its sibling has the minimum 1 key: merge [75 92]. The parent empties and [65] is minimal: merge [65 70]. Root becomes [50].

My answer

Q39. B-Trees Old Exams

Consider the B-tree of order 5: root [27]; level 1: [15 20] [33 45]; leaves: [10 12 14] [18 19] [21 23 26] [29 31] [38 42] [47 50]. The resulting tree will definitely have one level less than the current tree after deleting the key

(A) 45

(B) 27

(C) 20

(D) 18

Answer: (A). Replacing 45 by 42 or 47 empties a minimal leaf whose siblings are minimal: merge, then [33] underflows and [15 20] is minimal: merge with the root key. 27 replaced by its predecessor 26 causes no underflow; 20 and 18 are fixed by rotation.

My answer

Q40. Quiz 4 Ver A (T251)

For the same B-tree of order 5 (root [27]; level 1: [15 20] [33 45]; leaves: [10 12 14] [18 19] [21 23 26] [29 31] [38 42] [47 50]), delete the key 33, replacing it by its **predecessor**. The result is

(A) root [15 20 27 38]; leaves: [10 12 14] [18 19] [21 23 26] [29 31] [42 45 47 50]

(B) root [27]; level 1: [15 20] [31 45]; leaves: [10 12 14] [18 19] [21 23 26] [29] [38 42] [47 50]

(C) root [15 20 27 45]; leaves: [10 12 14] [18 19] [21 23 26] [29 31 38 42] [47 50]

(D) root [27]; level 1: [15 20] [45]; leaves: [10 12 14] [18 19] [21 23 26] [29 31 38 42] [47 50]

Answer: (C). 31 replaces 33; [29] underflows and its sibling [38 42] is minimal: merge [29 31 38 42]. [45] underflows and [15 20] is minimal: merge [15 20 27 45], the new root.

My answer

Part B: Programming

All questions use this node class (order m; the arrays have one extra slot so a node can temporarily hold m keys during an overflow):

```
class BTreeNode {
    int[] key;          // key[0..n-1], sorted
    int n;              // number of keys in the node
    BTreeNode[] child;  // child[0..n]
    boolean isLeaf;
    BTreeNode(int m, boolean isLeaf) {
        key = new int[m];
        child = new BTreeNode[m + 1];
        this.isLeaf = isLeaf;
    }
}
```

Q41. [new]

Write a recursive method `static BTreeNode search(BTreeNode x, int k)` that follows the slides' `BTree-Search`: it returns the node that contains key `k`, or `null` if `k` is not in the subtree rooted at `x`.

Answer:

```
static BTreeNode search(BTreeNode x, int k) {
    if (x == null) return null;
    int i = 0;
    while (i < x.n && k > x.key[i]) i++;
    if (i < x.n && k == x.key[i]) return x;
    if (x.isLeaf) return null;
    return search(x.child[i], k);
}
```

My answer

Q42. [new]

Write a method `static int countKeys(BTreeNode x)` that returns the total number of keys stored in the B-tree rooted at `x` (0 for an empty tree).

Answer:

```
static int countKeys(BTreeNode x) {
    if (x == null) return 0;
    int total = x.n;
    if (!x.isLeaf)
```

```
        for (int i = 0; i <= x.n; i++) total += countKeys(x.child[i]);
    return total;
}
```

My answer

Q43. [new]

Write a method `static int height(BTreeNode root)` that returns the height of a B-tree (a single leaf root has height 0, an empty tree -1). Use the fact that all leaves of a B-tree are on the same level, so the method must run in $O(h)$ time.

Answer:

```
static int height(BTreeNode x) {
    if (x == null) return -1;
    int h = 0;
    while (!x.isLeaf) { x = x.child[0]; h++; }
    return h;
}
```

My answer

Q44. [new]

Write two methods `static int findMin(BTreeNode root)` and `static int findMax(BTreeNode root)` that return the smallest and the largest key in a non-empty B-tree.

Answer:

```
static int findMin(BTreeNode root) {
    BTreeNode x = root;
    while (!x.isLeaf) x = x.child[0];
```

```
    return x.key[0];
}
static int findMax(BTreeNode root) {
    BTreeNode x = root;
    while (!x.isLeaf) x = x.child[x.n];
    return x.key[x.n - 1];
}
```

My answer

Q45. [new]

Write a method `static void inorder(BTreeNode x, List<Integer> out)` that appends all keys of the B-tree rooted at `x` to `out` in ascending order (subtree T_0, k_1, T_1, k_2, ...).

Answer:

```
static void inorder(BTreeNode x, List<Integer> out) {
    if (x == null) return;
    for (int i = 0; i < x.n; i++) {
        if (!x.isLeaf) inorder(x.child[i], out);
        out.add(x.key[i]);
    }
    if (!x.isLeaf) inorder(x.child[x.n], out);
}
```

My answer

Q46. [new]

During deletion, a key in a non-leaf node is swapped with its predecessor or successor. Write `static int predecessor(BTreeNode x, int i)` and `static int successor(BTreeNode x, int i)` that return

the predecessor and the successor of `x.key[i]`, where `x` is a non-leaf node (predecessor: one step left, then all the way right; successor: one step right, then all the way left).

Answer:

```
static int predecessor(BTreeNode x, int i) {
    BTreeNode p = x.child[i];            // one step to the left
    while (!p.isLeaf) p = p.child[p.n];  // then all the way right
    return p.key[p.n - 1];
}
static int successor(BTreeNode x, int i) {
    BTreeNode s = x.child[i + 1];        // one step to the right
    while (!s.isLeaf) s = s.child[0];    // then all the way left
    return s.key[0];
}
```

My answer

Q47. [new]

Write a method `static boolean validCounts(BTreeNode root, int m)` that checks the key-count rules of a B-tree of order `m`: the root has between 1 and $m-1$ keys and every other node has between $\lceil m/2 \rceil - 1$ and $m-1$ keys.

Answer:

```
static boolean validCounts(BTreeNode root, int m) {
    if (root == null) return true;
    if (root.n < 1 || root.n > m - 1) return false;
    if (!root.isLeaf)
        for (int i = 0; i <= root.n; i++)
            if (!validNonRoot(root.child[i], m)) return false;
    return true;
}
static boolean validNonRoot(BTreeNode x, int m) {
    int min = (m + 1) / 2 - 1;           // ceil(m/2) - 1
    if (x.n < min || x.n > m - 1) return false;
    if (!x.isLeaf)
        for (int i = 0; i <= x.n; i++)
            if (!validNonRoot(x.child[i], m)) return false;
    return true;
}
```

My answer

Q48. [new]

Write a method `static boolean leavesSameLevel(BTreeNode root)` that returns `true` if all leaves of the tree are on the same level (and no internal node has a missing child), as required in a B-tree.

Answer:

```
static boolean leavesSameLevel(BTreeNode root) {
    if (root == null) return true;
    int h = 0;                                    // depth of the leftmost leaf
    for (BTreeNode x = root; !x.isLeaf; x = x.child[0]) h++;
    return check(root, 0, h);
}
static boolean check(BTreeNode x, int level, int h) {
    if (x.isLeaf) return level == h;
    for (int i = 0; i <= x.n; i++)
        if (x.child[i] == null || !check(x.child[i], level + 1, h))
            return false;
    return true;
}
```

My answer

Q49. [new]

Insertion in a B-tree of **odd** order `m`: after a key is inserted, `parent.child[i]` holds `m` keys (overflow).

Write `static void splitChild(BTreeNode parent, int i, int m)` that splits that child into a left node (first $\lfloor m/2 \rfloor$ keys) and a right node (last $\lfloor m/2 \rfloor$ keys) and moves the middle key up into `parent` at position `i` (assume `parent` has room for one more key). Children of an internal node must be split as well.

Answer:

```
static void splitChild(BTreeNode parent, int i, int m) {
    BTreeNode full = parent.child[i];      // full.n == m, m odd
    int mid = m / 2;
    BTreeNode right = new BTreeNode(m, full.isLeaf);
    right.n = m - mid - 1;
    for (int j = 0; j < right.n; j++) right.key[j] = full.key[mid + 1 + j];
    if (!full.isLeaf)
        for (int j = 0; j <= right.n; j++) {
            right.child[j] = full.child[mid + 1 + j];
            full.child[mid + 1 + j] = null;
        }
    int w = full.key[mid];                 // middle key goes up
    full.n = mid;                          // left part stays in 'full'
    for (int j = parent.n; j > i; j--) {   // make room in parent
        parent.key[j] = parent.key[j - 1];
        parent.child[j + 1] = parent.child[j];
    }
    parent.key[i] = w;
    parent.child[i + 1] = right;
    parent.n++;
}
```

My answer

Q50. [new]

Deletion case 2: `parent.child[i]` has underflowed and its right sibling `parent.child[i+1]` has at least $\lceil m/2 \rceil$ keys. Write `static void rotateLeft(BTreeNode parent, int i)` that performs the left key-rotation of the slides: (1) move the separating key `parent.key[i]` to the end of the underflowed node, (2) move the minimum key of the right sibling up to where the separator was, (3) make the old leftmost subtree of the sibling the new rightmost subtree of the underflowed node.

Answer:

```
static void rotateLeft(BTreeNode parent, int i) {
    BTreeNode cur = parent.child[i], sib = parent.child[i + 1];
    cur.key[cur.n++] = parent.key[i];          // separator goes down
    parent.key[i] = sib.key[0];                // min of sibling goes up
    if (!sib.isLeaf) cur.child[cur.n] = sib.child[0];
    for (int j = 0; j < sib.n - 1; j++) sib.key[j] = sib.key[j + 1];
    if (!sib.isLeaf)
        for (int j = 0; j < sib.n; j++) sib.child[j] = sib.child[j + 1];
    sib.child[sib.n] = null;
    sib.n--;
}
```

My answer